Processos de Software

Dívida técnica

DIM0510 — Turma 01 · Sprint 1 · aula de 07/10

Prof. Fernando · UFRN · 2026.2

Avisos

	Antes	Agora
Prova escrita	09/11 (segunda)	**11/11 (quarta)**, em laboratório, no horário da aula
09/11 (segunda)	presencial: prova	🔵 online: acompanhamento de projetos
11/11 (quarta)	online: acompanhamento	🟢 presencial: **prova escrita**
Prova de reposição	02/12	02/12 (sem mudança)

- Vale no lugar do calendário dos slides 06; o conteúdo e as regras da prova são os mesmos
- Entrega da Sprint 1: **16/10 (sexta), 23:59**
- Próximo encontro: 14/10, online, acompanhamento de projetos

Roteiro de hoje

Min	Bloco	Pergunta
3	Avisos	—
8	A metáfora	Por que chamar de dívida? O que são o principal e os juros?
8	Tipos de dívida	Toda dívida é descuido?
6	Consequências	O que acontece quando ela não é paga?
10	Como tratar	Como evitar, tornar visível e pagar?
15	Atividade em grupo	Quais são as dívidas do nosso projeto?

> A aula liga três assuntos já vistos: as práticas de XP (slides 03), as métricas de fluxo (slides 05) e a retrospectiva (slides 06).

A metáfora

> "Entregar um código pela primeira vez é como contrair uma dívida. Uma pequena dívida acelera o desenvolvimento, desde que seja paga logo, com uma reescrita. [...] O perigo aparece quando a dívida não é paga. Cada minuto gasto em código que não está bem certo conta como juros sobre essa dívida."
>
> Ward Cunningham, 1992 (tradução livre)

- Cunningham usou a imagem para explicar à equipe e aos gestores por que entregar cedo, para aprender com o uso, era uma boa decisão
- A condição vinha junto: o código precisa ser reescrito à medida que a equipe entende melhor o domínio
- A dívida, na origem, é um recurso de quem aprende rápido; o problema é deixar de pagar

📖 **Ref.** Ward Cunningham — The WyCash Portfolio Management System

Principal e juros

Termo	Na dívida financeira	No software
Principal	o valor emprestado	o trabalho para deixar o código como deveria estar
Juros	o que se paga a mais enquanto a dívida existe	o esforço a mais em cada mudança feita sobre esse código

Um exemplo com números (Fowler):

```
Funcionalidade nova num módulo bem estruturado ........ 4 dias
A mesma funcionalidade no módulo confuso ............... 6 dias   → 2 dias de juros
Reorganizar o módulo ................................... 5 dias   → o principal
```

- Para uma funcionalidade só, pagar o principal (5 dias) custa mais que os juros (2 dias)
- Com três funcionalidades previstas nesse módulo, os juros somam 6 dias: pagar compensa
- Código que ninguém vai alterar não cobra juros

📖 **Ref.** Martin Fowler — Technical Debt

Como a dívida aparece

Sintoma	Exemplo num projeto da disciplina
Projeto que fazia sentido e deixou de fazer	a estrutura da Sprint 0 não comporta as telas ou rotas novas
Defeitos conhecidos e não corrigidos	"esse erro acontece às vezes, depois a gente vê"
Cobertura de testes insuficiente	regra de negócio sem nenhum teste
Teste manual onde caberia teste automatizado	conferir tudo à mão antes de cada entrega
Integração e implantação manuais	só um integrante sabe montar e publicar
Conhecimento concentrado	só um integrante entende um módulo

- Hoje o termo cobre vários problemas diferentes; por isso vale nomear o tipo antes de discutir
- Nem tudo é código: decisão não documentada e processo que ninguém segue também cobram juros

Três tipos de dívida

Tipo	Como surge	O que fazer
Ingênua (não intencional)	falta de prática, de testes ou de cuidado com o projeto do código	evitar: boas práticas de engenharia e revisão
Inevitável	o entendimento do produto evolui; uma biblioteca externa muda	aceitar e pagar aos poucos, conforme se aprende
Estratégica	decisão consciente de tomar um atalho para cumprir um objetivo de curto prazo	registrar a decisão e o plano de pagamento

- A ingênua é a única que não traz benefício; as outras duas fazem parte de desenvolver software
- A estratégica só é estratégica quando a equipe sabe o que deve e quando pretende pagar

📖 **Ref.** RUBIN, Kenneth. *Essential Scrum*. Addison-Wesley, 2012. Capítulo 8.

O quadrante da dívida

	Deliberada	Não deliberada
Imprudente	"Não temos tempo para pensar no projeto do código"	"O que é separação em camadas?"
Prudente	"Precisamos entregar agora e lidar com as consequências depois"	"Agora sabemos como deveríamos ter feito"

- A distinção que importa é entre prudente e imprudente, mais do que entre dívida e "bagunça"
- Prudente e deliberada: o ganho de entregar antes compensa os juros, e a equipe sabe disso
- Prudente e não deliberada: acontece com equipes boas; só se descobre o melhor desenho depois de construir
- Imprudente e deliberada: a equipe conhece as boas práticas e decide ignorá-las; costuma atrasar mais do que adianta

📖 **Ref.** Martin Fowler — Technical Debt Quadrant

O que acontece quando a dívida cresce

Consequência	O que a equipe percebe
Crescimento não linear	a partir de certo ponto, mudanças pequenas passam a ter resultado incerto
Entregas mais lentas	o atalho de hoje reduz a velocidade de amanhã; o tempo entre entregas aumenta
Mais defeitos	o código fica complexo, e corrigir falhas toma o tempo das funcionalidades
Estimativas ruins	ninguém consegue prever quanto uma mudança leva
Perda de confiança	a equipe deixa de cumprir o que combina, e os clientes deixam de contar com ela
Produto parado	sem novidades nem correções, o produto perde usuários

> Nas métricas que vocês já coletam: o *cycle time* sobe sem que as tarefas tenham ficado maiores, e as falhas de CI e o retrabalho aumentam.

Como tratar (1): evitar a dívida ingênua

As práticas de XP existem, em boa parte, para isso:

Prática	Dívida que evita
Testes automatizados (TDD)	regra sem teste; medo de alterar o código
Refatoração	projeto do código que não acompanha o que a equipe aprendeu
Revisão de código e programação em par	conhecimento concentrado; solução que só o autor entende
Integração contínua	integração demorada e sujeita a erro
Definição de pronto	"terminado", mas sem teste, sem revisão ou sem documentação

- Uma definição de pronto fraca deixa passar dívida a cada cartão concluído
- O limite de WIP ajuda: com menos trabalho aberto ao mesmo tempo, sobra atenção para terminar bem

Como tratar (2): tornar a dívida visível

Dívida que não está escrita não entra no planejamento.

```
Título:     Validação de tarefa duplicada nas duas telas de cadastro
Tipo:       inevitável (surgiu quando a segunda tela foi criada)
Onde:       TelaCadastro.kt e TelaEdicao.kt
Juros:      toda regra nova precisa ser escrita duas vezes; já houve divergência
Principal:  cerca de 3 horas para extrair uma função comum e testá-la
```

- Uma issue por dívida, com a etiqueta `divida-tecnica`, no mesmo quadro do restante do trabalho
- O campo que mais importa é o dos juros: o que essa dívida atrasa ou arrisca?
- Para quem decide prioridades, vale falar em consequências (prazo, defeitos), não em detalhes de código

📖 **Ref.** GitHub — Managing labels

Como tratar (3): pagar

- **Deixar o código melhor do que encontrou.** Ao mexer num trecho para uma funcionalidade, aproveitar para arrumá-lo: a dívida é paga onde o código mais muda
- **Pagar primeiro a de juros altos.** Prioridade para o código alterado com frequência e para o que causa defeitos
- **Pagar junto com trabalho de valor.** Reservar parte de cada sprint rende mais que uma "sprint de limpeza" sem entrega
- **Pagar aos poucos.** Passos pequenos, cada um com os testes passando

Nem toda dívida precisa ser paga:

Situação	Por quê
Código que não será mais alterado	não cobra juros
Protótipo descartável	será jogado fora
Produto perto do fim da vida	o investimento não retorna

📖 **Ref.** Martin Fowler — Opportunistic Refactoring

No MUSI: dívida de processo

A retrospectiva do MUSI (slides 06) encontrou dívidas que não estão no código:

Dívida (situação em 29/09)	Tipo	Juros	Pagamento previsto
Quadro parado, com três cartões em `Todo`	ingênua	ninguém sabe o estado do trabalho sem perguntar	ação A1: atualizar o quadro
`main` aceita integração sem pull request	ingênua	os dois PRs integrados não tiveram revisão	ação A2: exigir PR (feita em parte)
A verificação local (`./verificar.sh`) depende de alguém lembrar de rodar	ingênua	falhas de CI que o script já teria apontado	ação A3: gancho `pre-push` em `.githooks/`

- As ações da retrospectiva são o plano de pagamento: têm responsável, prazo e forma de verificar
- Dez dias depois, só a ação mecânica tinha andado: dívida sem dono e sem data tende a ficar

Atividade em grupo — 15 minutos

1. **Levantar (5 min).** Cada integrante aponta uma dívida do projeto do grupo: no código, nos testes, na documentação ou no processo
2. **Classificar (3 min).** Para cada uma: ingênua, inevitável ou estratégica? Em que quadrante?
3. **Estimar (4 min).** Quais são os juros (o que ela atrasa ou arrisca) e o principal (quanto custa pagar)?
4. **Registrar (3 min).** Abrir uma issue para as três de juros mais altos, com a etiqueta `divida-tecnica`

```
gh label create divida-tecnica --description "Dívida técnica registrada" --color D93F0B
gh issue create --label divida-tecnica --title "..." --body "Tipo: ... Onde: ... Juros: ... Principal: ..."
```

Ao final, cada grupo diz em uma frase qual dívida pagaria primeiro, e por quê.

Como isto entra na entrega de 16/10

Não há critério novo na rubrica. O que foi levantado hoje serve a duas partes da entrega:

Parte da entrega	Como usar
`docs/retrospectiva-01.md`	uma dívida de juros altos pode virar um dos fatos e uma das ações, com responsável, prazo e verificação
Quadro no GitHub Projects	as issues com `divida-tecnica` ficam no backlog, visíveis, ao lado das histórias

- Na Sprint 2, a integração contínua e as métricas DORA dão números para acompanhar essas dívidas
- Registrar uma dívida não baixa a nota: mostra que o grupo conhece o estado do próprio projeto

Onde estudar depois

- Leitura: `leituras/processos-s1-divida-tecnica.md` — o conteúdo desta aula por extenso, com exercícios e dúvidas frequentes
- Martin Fowler — Technical Debt: a definição, o exemplo com números e quando pagar
- Martin Fowler — Technical Debt Quadrant: prudente × imprudente, deliberada × não deliberada
- Martin Fowler — Is High Quality Software Worth the Cost?: por que qualidade interna reduz o custo, e não o contrário
- Ward Cunningham — The WyCash Portfolio Management System: o relato de 1992 em que a metáfora aparece
- RUBIN, Kenneth. *Essential Scrum*. Addison-Wesley, 2012. Capítulo 8: tipos, consequências e formas de gerenciar a dívida

Próximos encontros

- 14/10 (quarta): online, acompanhamento de projetos
- 16/10 (sexta), 23:59: **entrega da Sprint 1**
- 19 e 21/10: em sala, todo o conteúdo da Sprint 2 (DevOps, integração contínua, Docker, métricas DORA e mapeamento do fluxo de valor)